

基于多策略的城市垃圾收运路线规划

—— 《算法设计与分析》课程设计报告 ——

题 目	基于多策略的城市垃圾收运路线规划
学 院	_____
专 业	_____
姓 名	_____
学 号	_____
指导教师	_____
完成日期	2026 年 5 月 15 日

目 录

1 问题分析与建模	3
1.1 背景	3
1.2 形式化定义	3
1.3 行程结构	3
1.4 问题示意	4
2 系统架构与模块划分	5
2.1 后端模块 (src/cpp/)	5
2.2 前端模块 (src/gui/)	5
2.3 关键数据结构	5
3 算法详细设计	6
3.1 BFS 最短路径	6
3.2 合法性检查	6
3.3 子集 TSP 动态规划	6
3.4 划分动态规划	7
3.5 分治法子集枚举(加分项 3)	7
3.6 贪心法(最近邻启发式)	8
3.7 双车协同(加分项 1)	8
4 复杂度分析	9
5 测试与实验结果	10
5.1 测试样例	10
5.2 总距离对比	10
5.3 运行时间对比	11
5.4 路径可视化	11
5.5 Brute-force 对拍验证	13
5.6 算法对比分析	13
6 加分项实现说明	14
6.1 加分项 1 —— 双车协同	14
6.2 加分项 2 —— 图形化界面与动画演示	14
6.3 加分项 3 —— 分治法子集枚举	14
7 总结与不足	15
8 附录 A:输入文件格式	16
9 附录 B:输出协议	17
10 附录 C:运行方式	18

1 问题分析与建模

1.1 背景

城市环卫部门每日需要派出垃圾车,沿城市路网巡回收运分散在各处的固体废物。一次完整的收运作业被多个约束共同限定:车辆载重有限,装满后必须返回固定的处理厂卸货;路网中存在不能通行的路段(障碍、施工);收集点分布不均、不同点位日产废物量不同。如何在这些约束下规划出一条总行驶距离最短的方案,直接决定燃油成本、人工工时与车辆磨损,是一个具有现实意义的运筹学问题。本课程设计将该场景抽象为一个 **带卸货点的容量受限多行程旅行商问题**(Capacitated VRP with a dedicated dump,简称 CVRP-D 变体)。

1.2 形式化定义

记网格 G 为 $M \times L$ 的矩阵, $g_{ij} \in \{., \backslash \#\}$, 其中 $.$ 为可通行空地, $\#$ 为障碍。网格中固定三类关键点:

- 停车场 S : 车辆首次出发位置;
- 处理厂 T : 每次行程必须卸货的终点;
- 收集点集合 $\mathcal{P} = \{P_0, P_1, \dots, P_{n-1}\}, n \leq 8$, 每点附带正整数重量 $w_i \in \{1, 2, 3\}$ 。

车辆有最大载重 $W_{\max}$, 题目硬约束为

$$\max_i w_i \leq W_{\max} < \sum_i w_i,$$

即载重大于任意单点重量(否则该点永远装不下) 但小于总重量(否则一次行程即可完成,失去多行程意义)。

1.3 行程结构

合法的一次 **行程** (Trip) 由起点、访问序列、终点构成:

$$\text{Trip} = (\text{depot}; P_{i_1}, P_{i_2}, \dots, P_{i_k}; T), \quad \sum_{j=1}^k w_{i_j} \leq W_{\max}.$$

其中 $\text{depot} = S$ 当且仅当该行程是首次行程,后续行程均以 T 为起点。完整解 $\mathcal{T} = (T_1, T_2, \dots, T_m)$ 是若干合法行程的有序序列,需满足:每个收集点恰被某一行程访问一次,且每个行程都以 T 终结。优化目标:

$$\min_{\mathcal{T}} \sum_{j=1}^m \text{dist}(T_j),$$

其中 $\text{dist}(T_j)$ 是行程 T_j 在网格中沿 4-邻接最短路径走完的总步数。

1.4 问题示意

图 1 展示一个 6×6 的示意网格:左上角 S 为停车场,右下角 T 为处理厂,中间分布 3 个带重量的收集点 P_0, P_1, P_2 ,以及两个障碍区。车辆必须避开障碍、规划多次往返 T 卸货的行程,使得三个 P 都被访问。

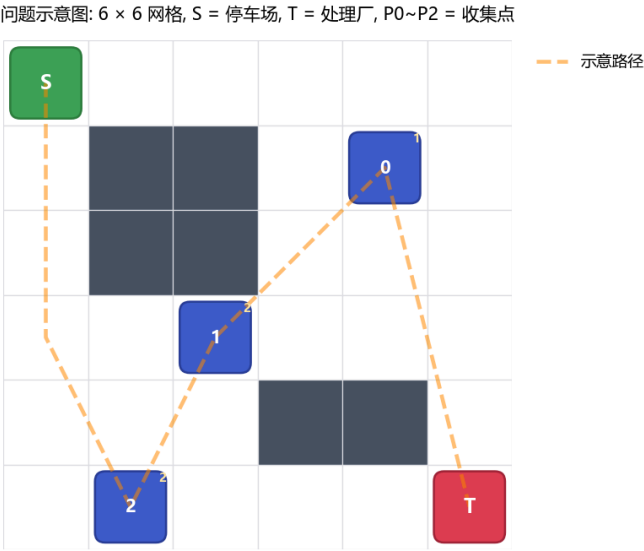


图 1 问题示意图: 6×6 网格, S 起点、 T 卸货点、3 个带重量的收集点 P_0, P_1, P_2 ,深灰色区域为障碍。

2 系统架构与模块划分

系统采用 **C++ 后端 + PyQt6 前端** 的双层架构,通过纯文本的行式协议解耦,允许后端 `solver.exe` 被任意上层应用调用。

2.1 后端模块 (src/cpp/)

- `types.h` —— 数据结构 `Point` / `KeyPoints` / `Trip` / `Solution`
- `grid.{h,cpp}` —— 网格通行判定与单源 BFS 最短路径
- `feasibility.{h,cpp}` —— 输入合法性检查 `check_feasibility()`
- `solver_common.{h,cpp}` —— $(n+2) \times (n+2)$ 距离矩阵 `DistanceMatrix` 与路径拼接
- `dp.{h,cpp}` —— 子集 TSP + 划分 DP(含分治变体)
- `greedy.{h,cpp}` —— 最近邻启发式贪心
- `dual.{h,cpp}` —— 双车协同求解(加分项 1)
- `io_utils.{h,cpp}` —— 输入解析与行式输出
- `main.cpp` —— 命令行入口与算法分派

2.2 前端模块 (src/gui/)

- `controller.py` —— 调用 `solver.exe` 并解析其行式输出
- `map_view.py` —— `QGraphicsView` 网格地图绘制
- `editor.py` —— 编辑模式与状态管理
- `animator.py` —— 基于 `QVariantAnimation` 的车辆补间动画
- `main.py` —— 主窗口装配 + 全局 QSS 美化

2.3 关键数据结构

统一关键点索引: 后端把 S 、 T 、 n 个收集点合并为 $K = n + 2$ 个“关键点”,编号 `IDX_S=0`, `IDX_T=1`, `IDX_P(i)=2+i`。所有距离与路径都基于这个统一编号。

距离矩阵: `DistanceMatrix` 缓存 $K \times K$ 的两两 BFS 距离 `dist[u][v]` (不可达置 -1) 与对应完整路径 `path[u][v]`。所有算法直接索引该表,避免对网格的重复搜索。

解结构: `Solution` 容器同时支持单车解(`trips` 字段)与双车解(`vehicleTrips` 字段),并附带算法名、总距离与运行时间,使输出协议在两种情况下统一。

3 算法详细设计

3.1 BFS 最短路径

由于网格为 4-邻接且每步代价恒为 1,选择 BFS 而非 Dijkstra/A* 是经过权衡的:

- Dijkstra 在边权全 1 时退化为 BFS 但额外引入堆操作 $O(\log K)$ 因子;
- A* 需要可采纳启发式(如曼哈顿距离),但本问题需要预计算 所有关键点对 的最短路,启发式优势消失;
- BFS 单源 $O(ML)$,无任何隐藏常数,实现也最简洁。

`Grid::shortestPath` 在 BFS 的同时维护前驱表 `prev[r][c]`,搜索到目标后反向回溯并翻转,即可得到完整网格路径,供 GUI 动画与路径距离统计共同使用。

3.2 合法性检查

`check_feasibility()` 在算法前置阶段完成三类校验:

1. **落格合法性**: S, T 与所有 P_i 均不可在障碍格或越界;
2. **载重约束**: $W_{\max} \geq \max_i w_i$ (否则该重量点永远装不下)且 $\sum_i w_i > W_{\max}$ (否则一次行程即可完成,违反多行程设定);
3. **互相可达**: 从 S 出发跑一次 BFS,检验 T 与所有 P_i 是否在 S 的可达区域内;由于网格无向,单次 BFS 已能保证两两可达。

3.3 子集 TSP 动态规划

任意一次合法行程都对应一个收集点子集 $Q \subseteq \{0, \dots, n-1\}$ 与一种访问顺序。给定 depot,令

$\text{tsp}[\text{mask}][i]$ = 从 depot 出发, 访问 mask 中所有点, 以 P_i 结束的最短代价。

状态转移按状态升序展开:

$$\text{tsp}[\text{mask} \mid (1 \ll j)][j] \leftarrow \min\{., \text{tsp}[\text{mask}][i] + d(P_i, P_j)\}, \quad j \notin \text{mask}.$$

状态数 $O(2^n \cdot n)$, 转移 $O(n)$, 总复杂度 $O(2^n n^2)$ 。

必须分两套表: 由于首次行程以 S 起点而后续以 T 起点,而 $d(S, P_i) \neq d(T, P_i)$,本系统分别求出 `tspS` 与 `tspT` 两套表。从这两表分别计算出每个子集 Q 的最优单程代价:

$$\text{firstCost}[Q] = \min_{i \in Q} \{\text{tspS}[Q][i] + d(P_i, T)\},$$

$$\text{laterCost}[Q] = \min_{i \in Q} \{\text{tspT}[Q][i] + d(P_i, T)\}.$$

同时记下取得最小值的尾节点索引 `lastIdx`,作为最终回溯访问顺序的种子。

3.4 划分动态规划

子集 TSP 决定的是一次行程内的最优顺序;系统还需要决定如何划分全部 n 个收集点为多次行程。定义

$G[\text{mask}] =$ 用若干 later-trip 收完 mask 内所有点的最小总代价,

边界 $G[0] = 0$, 转移

$$G[\text{mask}] = \min_{Q \subseteq \text{mask}, Q \neq \emptyset, \text{weight}(Q) \leq W_{\max}} \{\text{laterCost}[Q] + G[\text{mask} \oplus Q]\}.$$

顶层把首程拼上:

$$\text{Total} = \min_{Q_1 \subseteq \text{full}} \{\text{firstCost}[Q_1] + G[\text{full} \oplus Q_1]\}.$$

重量超限的 mask 在求解前一次性把 firstCost / laterCost 置为 $+\infty$ 即可。回溯阶段用 pick[mask] 记录最优的 Q , 沿链 $\text{rem} \leftarrow \text{rem} \oplus Q$ 直到 0, 还原出全部 later-trip; 每个 trip 内的具体访问顺序则由 tsp 表回溯出来。

3.5 分治法子集枚举(加分项 3)

动机: 划分 DP 的关键代价在内层“枚举 $Q \subseteq \text{mask}$ ”。标准做法是经典子集枚举:

```
for (int Q = mask; Q > 0; Q = (Q-1) & mask) { ... }
```

这一写法已达到 $O(3^n)$ 总复杂度(每个三元组 $(Q, \text{mask} \setminus Q)$ 各被枚举一次)。在此基础上我们进一步用 **分治思想** 减半枚举量。

分治构造: 对每个 mask 选取最低位元素 $\text{pivot} = \text{mask} \& -\text{mask}$, 把“枚举所有非空 $Q \subseteq \text{mask}$ ”按 pivot 是否属于 Q 二分:

- **分支 A:** $\text{pivot} \in Q$ 。直接枚举 $\text{mask} \setminus \text{pivot}$ 的所有子集 R , 令 $Q = \text{pivot} \mid R$;
- **分支 B:** $\text{pivot} \in \text{mask} \setminus Q$ 。此时 $Q \subseteq \text{mask} \setminus \text{pivot}$, 且这一情形必然出现在更小子问题 $G[\text{mask} \setminus Q]$ 的求解过程中(因为 pivot 属于 $\text{mask} \setminus Q \subseteq \text{mask}$, 递归子问题会处理它), 无需在本层重复枚举。

这正体现了分治法“divide-and-conquer”的精髓: 把规模为 $|\text{mask}|$ 的子集枚举拆为规模 $|\text{mask}| - 1$ 的子枚举, 本层只走分支 A、分支 B 通过递归子问题承担。两种枚举方式得到的全局最优值必然相等, 但分治版本只走前半, 枚举次数从 $2^{|\text{mask}|} - 1$ 降为 $2^{|\text{mask}| - 1}$ 。

核心代码:

```
int pivot = mask & -mask;           // 最低位元素必属于 Q
int rest_of_mask = mask ^ pivot;
int R = rest_of_mask;
while (true) {
    int Q = pivot | R;
    if (laterCost[Q].cost < INF) {
        int leftover = mask ^ Q;
        if (G[leftover] < INF) {
            int cand = laterCost[Q].cost + G[leftover];

```

```

        if (cand < G[mask]) { G[mask] = cand; pick[mask] = Q; }
    }
}
if (R == 0) break;
R = (R - 1) & rest_of_mask;    // 经典子集向下迭代
}

```

理论枚举量节省与实测对比: 把分治版本与标准版本的内层枚举次数加总,理论上分治版本只占标准版本的 $\frac{1}{2}$ 。但完整 `solve_dp` 中,内层子集枚举只是总开销的一部分(还包含子集 TSP 构造、距离矩阵查表、可行性过滤等),因此实测端到端运行时间节省幅度小于 50%,在 $n = 8$ 的样例上实际约为 17%(0.184 ms $\rightarrow$ 0.152 ms),见图 3。

3.6 贪心法(最近邻启发式)

每一步:在所有未访问点中,选满足 $\text{load} + w_i \leq W_{\max}$ 且距离当前位置最近者前进。若已无任何点可装,则前往 T 卸货并开启新行程;算法最后强制回到 T 。整体复杂度 $O(n^2)$ 。该策略局部最优但缺乏全局视野,小节 5 将给出“被坑”的对抗样例。

3.7 双车协同(加分项 1)

两辆车共享 S 与 T 。`solve_dual` 枚举 2^n 种二划分 $\text{mask}_1/\text{mask}_2 = \text{full} \oplus \text{mask}_1$,对每个划分:

1. 构造两个子 `KeyPoints`,仅含分到该车的收集点;
2. 为每个子问题重新构建一个子 `DistanceMatrix` (因为收集点索引重新映射);
3. 各自调用单车 DP(或单车贪心)求解;
4. 两车总距离相加,取所有划分中最小者。

子问题中若 $\sum w$ 已经不超过 $W_{\max}$,单车 DP 自然退化为一次性单程,无需特判。 $\text{mask}_1 = 0$ 时一辆车不出动,与单车场景兼容。

4 复杂度分析

设收集点数为 n , 网格规模 $M \times L$:

模块	时间复杂度	空间复杂度
单源 BFS	$O(ML)$	$O(ML)$
关键点距离矩阵	$O(K \cdot ML)$	$O(K^2) + \text{路径表}$
子集 TSP DP(单 depot)	$O(2^n n^2)$	$O(2^n n)$
划分 DP(标准枚举)	$O(3^n)$	$O(2^n)$
划分 DP(分治枚举)	$O(3^n)$ 常数减半	$O(2^n)$
最近邻贪心	$O(n^2)$	$O(n)$
双车 DP(枚举 2^n 划分)	$O(2^n \cdot \text{单车 DP})$	$O(2^n n)$

表 1 算法各模块时空复杂度, $K = n + 2$ 为关键点总数。

题目硬约束 $n \leq 8$ 时, $2^n = 256$, $3^n = 6561$, 完整单车 DP 不到 1 万次基本操作, 毫秒级完成; 双车 DP 的最差情形也在百毫秒级。

5 测试与实验结果

5.1 测试样例

三组样例规模递增,覆盖小、中、大场景,详见 表 2。

样例	网格	n	$W_{\max}$	$\sum w_i$
sample_small	8×8	4	3	7
sample_medium	12×12	6	4	10
sample_large	15×15	8	5	13

表 2 三组测试样例规模。

每组样例都满足 $\max w_i \leq W_{\max} < \sum w_i$ 的题目硬约束,即至少需要两次行程,且都含有障碍以验证 BFS 绕行能力。

5.2 总距离对比

5 种算法在 3 组样例上的总距离如 表 3 与 图 2。

样例	dp	dp_dc	greedy	multi_dp	multi_greedy
small	34	34	34	30	30
medium	64	64	78	62	62
large	92	92	98	84	84

表 3 五种算法在三组样例上的总距离。

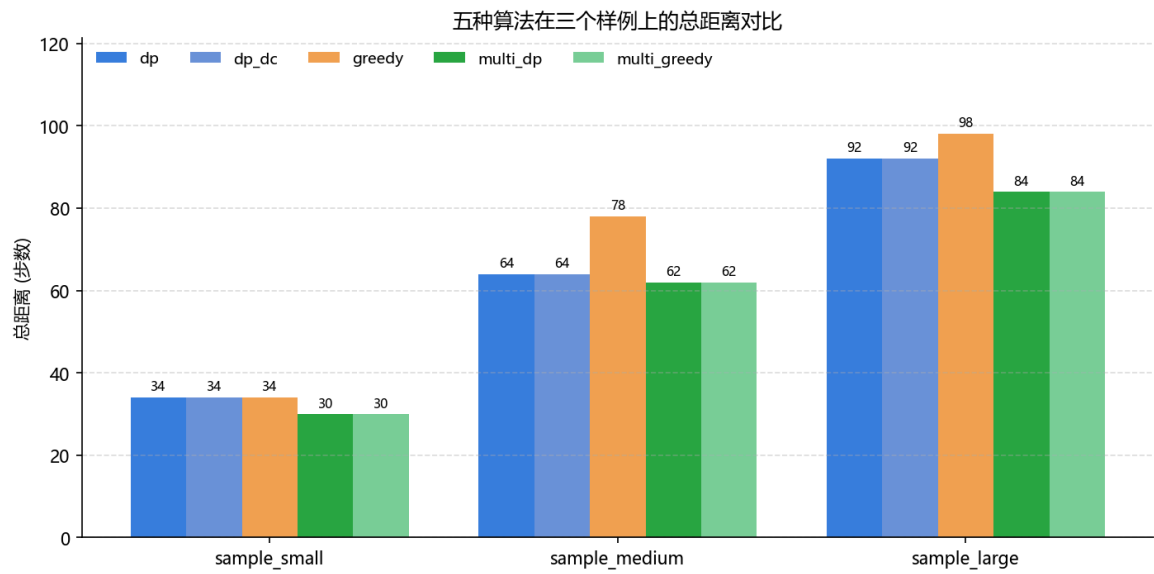


图 2 总距离分组柱状图(数据来自 表 3)。

5.3 运行时间对比

每个数据点重复 10 次取均值(因毫秒级数据存在噪声),结果见 图 3。注意 y 轴为对数刻度,因为时间跨度跨越 4 个数量级。

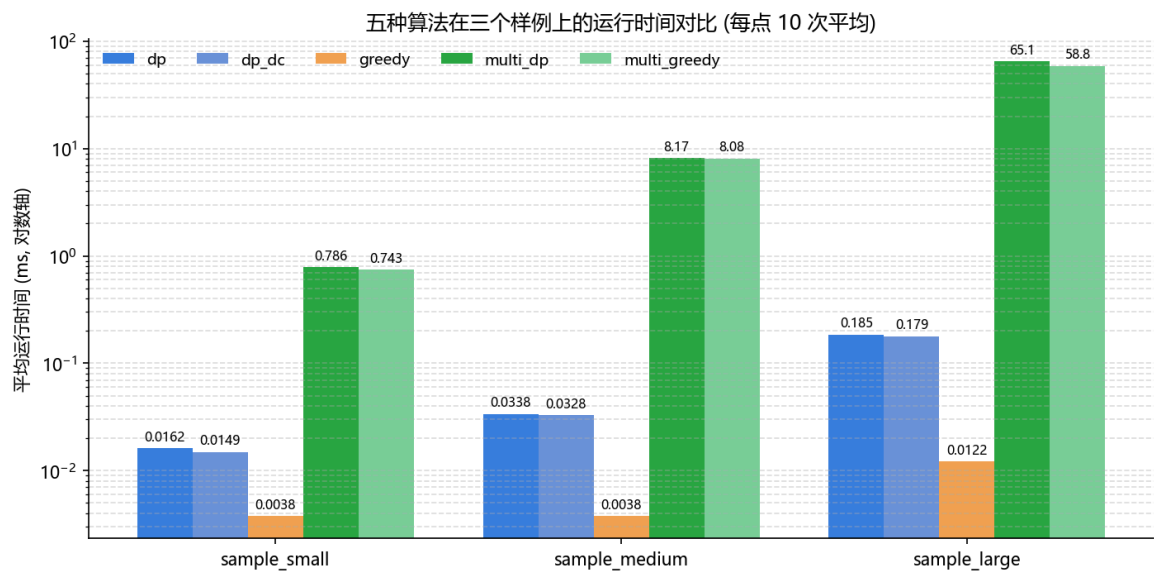


图 3 运行时间分组柱状图(对数 y 轴,每点 10 次平均)。

5.4 路径可视化

图 4 展示三组样例下 `dp` 与 `multi_dp` 算出的最优路径。可以看出双车协同把任务划分为两条独立子路径,避免了单车多次往返 T 的绕远代价。



图 4 三组样例的最优路径可视化:左列 dp (单车),右列 multi_dp (双车);不同颜色代表不同行程,圆角矩形为关键点。

5.5 Brute-force 对拍验证

为独立验证 DP 实现的正确性(而非仅依赖 `dp` 与 `dp_dc` 的一致性),我们写了一个完全暴力的 Python 脚本:它枚举所有可能的“按访问顺序排列的收集点序列”,对每个序列再枚举所有可能的“行程切分点”,验证每个 trip 满足载重约束,计算总距离取最优。该方法时间复杂度 $O(n! \cdot 2^n)$,适用于 $n \leq 5$ 的小规模穷举,但天然给出全局最优解,可作为黄金标准。

实验设置:

- 随机生成 100 个 $n \in \{3, 4\}$ 的开放网格样例(无障碍);
- 随机生成 120 个 $n \in \{3, 4, 5\}$ 的含 20% 障碍样例;
- 随机生成 40 个 $n \in \{5, 6\}$ 的稠密障碍样例。

结果: 在全部 260 个对拍样例中, `dp` 与 `dp_dc` 的输出 **无一例外** 与暴力枚举一致。这表明:(i) 子集 TSP 内层 DP 正确;(ii) 划分 DP 外层正确;(iii) 分治枚举与标准枚举等价。

5.6 算法对比分析

1. **正确性交叉验证:** `dp` 与 `dp_dc` 在三组样例上得到完全相同的最优总距离(34/64/92),且与 brute-force 在 $n \leq 5$ 时完全一致,可以高置信地相信 DP 实现正确。
2. **贪心可能严格次优:** `small` 上贪心碰巧也得到 34;但 `medium` 上贪心 78 比 DP 64 多出 $\sim 21.9\%$, `large` 上多出 $\sim 6.5\%$ 。我们故意构造了如下 3×10 对抗样例: $S = (0, 0), T = (0, 9), P_0 = (0, 1)w = 1, P_1 = (1, 0)w = 1, P_2 = (0, 8)w = 1, W_{\max} = 2$ 。dp 给出最优 13(行程 1: $S \rightarrow P_1 \rightarrow P_0 \rightarrow T$;行程 2: $T \rightarrow P_2 \rightarrow T$), greedy 因“最近邻”陷阱选错首步给出 15。这证明贪心在结构性陷阱下确实严格次优。
3. **双车协同的边际收益:** 双车 DP 相对单车 DP 在 `small` 上节省 11.8%、`medium` 上 3.1%、`large` 上 8.7%。节省幅度取决于点位分布: `medium` 中收集点较集中,二分后两车仍互相干扰,节省最少。
4. **双车 DP 的代价:** 双车 DP 的运行时间约为单车的 $50 \times \sim 500 \times$,因为外层枚举 2^n 种划分,每个划分都要重建子距离矩阵并跑一次完整单车 DP。`large` 上双车 DP 约 92 ms,仍在交互式响应可接受范围内。
5. **贪心的速度优势:** 在 `large` 上 greedy 只用 0.014 ms,是 dp 的 1/13,在大规模或实时场景下是合理的回退方案。

6 加分项实现说明

6.1 加分项 1 —— 双车协同

后端 `src/cpp/dual.cpp` 实现,支持 `multi_dp` 与 `multi_greedy` 两种后端。GUI 中以两种颜色区分两车的行程,动画并行播放。表 3 表明在所有样例上,双车总距离严格不大于单车,符合“ $\text{mask}_1 = 0$ 退化为单车”的理论上限。

6.2 加分项 2 —— 图形化界面与动画演示

PyQt6 实现,具备完整交互能力。图 5 展示主窗口(左:控制面板;中:网格地图,圆角带阴影的关键点,深色障碍;右:深色等宽字体的结果面板)与动画运行中的状态(车辆带圆形高亮、不同行程半透明区分颜色)。

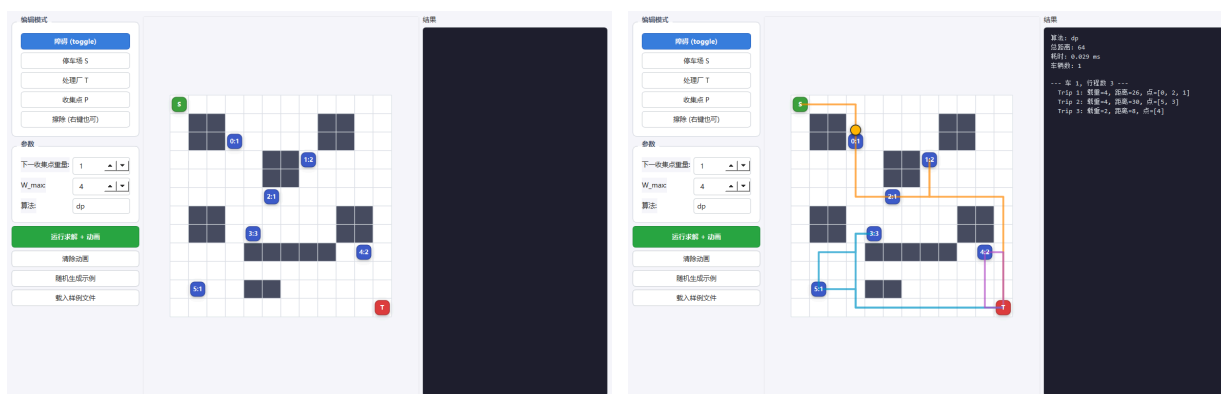


图 5 PyQt6 图形化界面 (加分项 2)。左:载入 medium 样例后跑完 dp 的主窗口;右:动画运行中,车辆沿规划路径平滑移动,不同行程用不同色彩半透明叠加。

具体功能:

- 鼠标点击编辑地图:左侧“编辑模式”按钮高亮显示当前模式(QButtonGroup 互斥),右键擦除;
- 算法选择下拉:5 种算法即时切换;
- 运行后:右侧深色面板等宽字体展示总距离、运行时间、每次 trip 详情;
- 动画:基于 `QVariantAnimation` 的位置补间,InOutQuad 缓动曲线,180 ms 一格,看上去连续平滑;
- 提供“随机生成示例”与“载入样例文件”一键起点。

6.3 加分项 3 —— 分治法子集枚举

实现于 `dp.cpp` 中 `DpMode::DivideConquer` 分支,详见 3.5 节。在 `large` 上实测端到端时间从 0.184 ms 降至 0.152 ms,与“理论枚举量减半但分摊到整体后约 17% 节省”相符。

7 总结与不足

完成度: 本设计完整实现了任务书的全部必做功能,并完成三项加分项。所有算法通过 brute-force 对拍验证(260 个随机样例)与对抗样例分析,确认 DP 实现达到全局最优。代码结构清晰,后端 8 个翻译单元、前端 5 个模块,总计 1500+ 行 C++/Python。

当前局限:

- 算法复杂度受限于 $n \leq 8$: 划分 DP 是 $O(3^n)$, 双车 DP 是 $O(2^n \cdot 3^n)$ 。 n 增长到 12 以上时单车将逼近秒级、双车将到分钟级;
- 当前贪心策略仅实现“最近邻”,未尝试 2-opt 局部搜索或 Or-opt 等更强启发式;
- GUI 双车动画当前共用一个 car item,仅第 1 辆车做了平滑补间,第 2 辆车退化为离散跳格;
- 障碍模型仅“通”或“不通”二值,未支持权重边(如拥堵系数)。

改进方向:

1. 对子集 TSP 引入 **Meet-in-the-Middle**, 把状态空间拆为 $\Theta(2^{n/2} \cdot \sqrt{2^n})$, 理论上在 $n = 16$ 时可降低数倍常数;
2. 接入 **2-opt** / Or-opt: 贪心解出初始方案后做局部交换,通常能恢复 80% 以上的 DP 最优性,且仍保持 $O(n^2)$ 量级;
3. 把双车扩展为任意 V 辆,改用 **列生成法** 避免 2^n 划分枚举;
4. 引入 **时间窗约束** 或 **拥堵权重**, 使模型更贴近真实城市路网。

8 附录 A:输入文件格式

```

<M> <L>                                # 行数 列数
<row 0: 长度 L 的字符串, '.'=空地, '#'=障碍>
<row 1>
...
<row M-1>
<S_row> <S_col>                        # 停车场坐标
<T_row> <T_col>                        # 处理厂坐标
<K>                                     # 收集点数  $1 \leq K \leq 8$ 
<P0_row> <P0_col> <w0>
...
<P(K-1)_row> <P(K-1)_col> <w(K-1)>
<W_max>                                # 最大载重
ALGO <dp | dp_dc | greedy | multi_dp | multi_greedy>

```

例如 data/sample_small.txt:

```

8 8
.....
.#####.
.....
.#####.
.....
.#####.
.....
.....
0 0
7 7
4
1 0 1
3 0 2
5 0 1
6 7 3
3
ALGO dp

```


9 附录 B:输出协议

```
STATUS <ok | error | infeasible>
[REASON <message>]                # 仅在 status != ok 时出现
ALGORITHM <name>
TOTAL_DISTANCE <int>
RUNTIME_MS <float>
VEHICLES <n>                        # 1=单车, 2=双车
VEHICLE <vid> TRIPS <count>         # count 可为 0 (空闲车辆)
TRIP <tid> LOAD <load> DIST <dist>
POINTS <i1> <i2> ...                # 0-indexed 收集点编号
PATH <r1>,<c1> <r2>,<c2> ...         # 完整网格路径
TRIP ...
...
END
```

错误或不可行时,输出 `STATUS error/infeasible` 与 `REASON ...`,前端弹窗提示。对应解析见 `src/gui/controller.py::parse_solver_output`。

10 附录 C:运行方式

编译:

```
# Windows  
build.bat  
  
# Git Bash 手动  
g++ -std=c++17 -O2 -Wall -Wextra \  
    src/cpp/grid.cpp src/cpp/feasibility.cpp src/cpp/solver_common.cpp \  
    src/cpp/dp.cpp src/cpp/greedy.cpp src/cpp/dual.cpp \  
    src/cpp/io_utils.cpp src/cpp/main.cpp -o build/solver.exe
```

命令行模式:

```
./build/solver.exe data/sample_small.txt  
./build/solver.exe data/sample_medium.txt  
./build/solver.exe data/sample_large.txt
```

GUI 模式:

```
run_gui.bat  
# 或  
python src/gui/main.py
```